Adequate & Explainable Information Capture for Software Generation

Daphne Jarabek, Dr. Jacques Carette, Dr. Spencer Smith, Jason Balaci
Department of Computing and Software, McMaster University
https://jacquescarette.github.io/Drasil

McMaster University | Computing & Software

Background

The Drasil research project aims to generate complete Scientific Computing Software repositories (including code, documentation, specifications, tests, build scripts, etc.) while avoiding duplicated information. This generation is done deterministically based on the input knowledge.

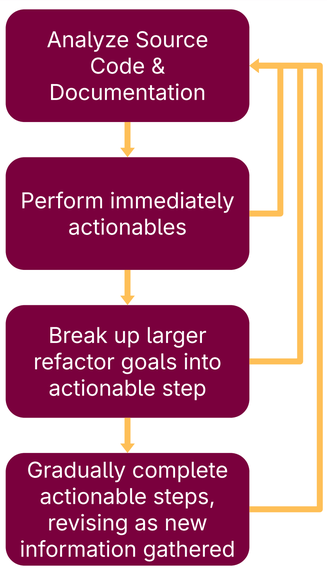
Problem

Drasil generates a host of physics-focused software artifacts. However, extension and usage of Drasil is complicated because our explanation of the generation process and rationale for the chunk design (i.e., knowledge organization) is confusing/incoherent/questionable.

Goal

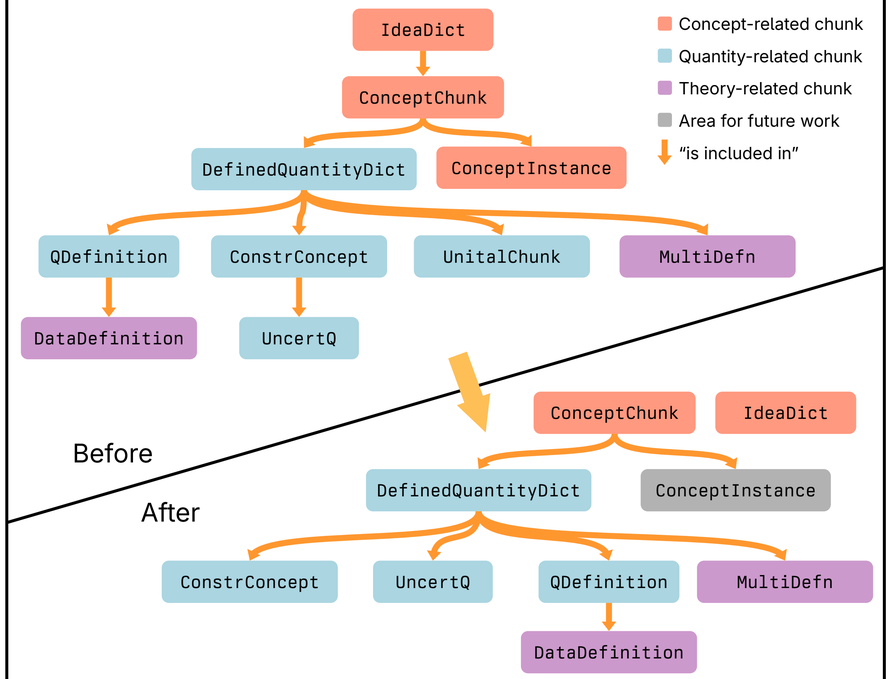
To refine Drasil's documentation, chunk graph, and codebase to be more explainable and allow for deeper knowledge capture.

Method



Immediately Actionables:
- Removing dead code
- Correcting typos/inaccuracies
- Mark code for removal with deprecated flags

Select Subset of Chunk Graph: Before & After



Sample Refactor

Context
A constrained concept has a unique id, an inner quantity, a list of constraints and an optional reasonable value.

```
data ConstrConcept =
  CC { _uu      :: UID
     , _defq    :: DefinedQuantityDict
     , _constr  :: [ConstraintE]
     , _reasV   :: Maybe ReasonableValue
     }
```

UncertQ Before
An uncertain quantity previously had a unique id, a constrained concept and an uncertainty.

```
data UncertQ =
  UQ { _uu    :: UID
     , _coco  :: ConstrConcept
     , _unc   :: Uncertainty
     }
```

UncertQ After
Now, the fields from ConstrConcept have been flattened into UncertQ, flattening the hierarchy.

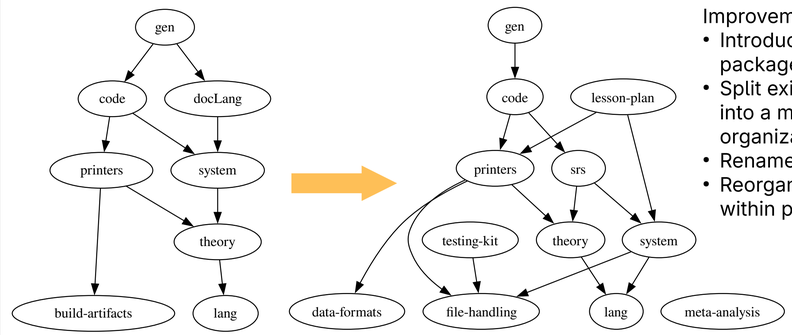
```
data UncertQ =
  UQ { _uu      :: UID
     , _defq    :: DefinedQuantityDict
     , _constr  :: [ConstraintE]
     , _reasV   :: Maybe ReasonableValue
     , _unc     :: Uncertainty
     }
```

Sample Quantity

Name	Definition
force	an interaction that tends to produce change in the motion of an object

Unique Identifier	Symbol	Unit	Space (Type)	Potential Formula
`"force"`	**F**	Newtons	`Vect Real`	`F = ma`

Select Subset of Drasil Package Graph: Before & After



Improvements:
- Introduced new packages
- Split existing packages into a more logical organization
- Renamed packages
- Reorganized code within packages

Results

- **>70** pull requests merged
- **>20** constructors and functions deleted
- **5** constructors marked for removal
- **1** unneeded chunk type deleted
- Clarified documentation
- New testing suites introduced

Future Work

- Incorporate units into type system
- Split ConceptInstance into more specific chunks each containing deeper knowledge.
- Improving chunk type names

Acknowledgements

I would like to acknowledge the support of my supervisors Dr. Jacques Carette, Dr. Spencer Smith, and Jason Balaci, and my colleagues Brandon Bosman and Camila Shibata. I would like to thank the Natural Sciences and Engineering Research Council of Canada (NSERC) and the Faculty of Engineering for funding my work this summer.

NSERC CRSNG